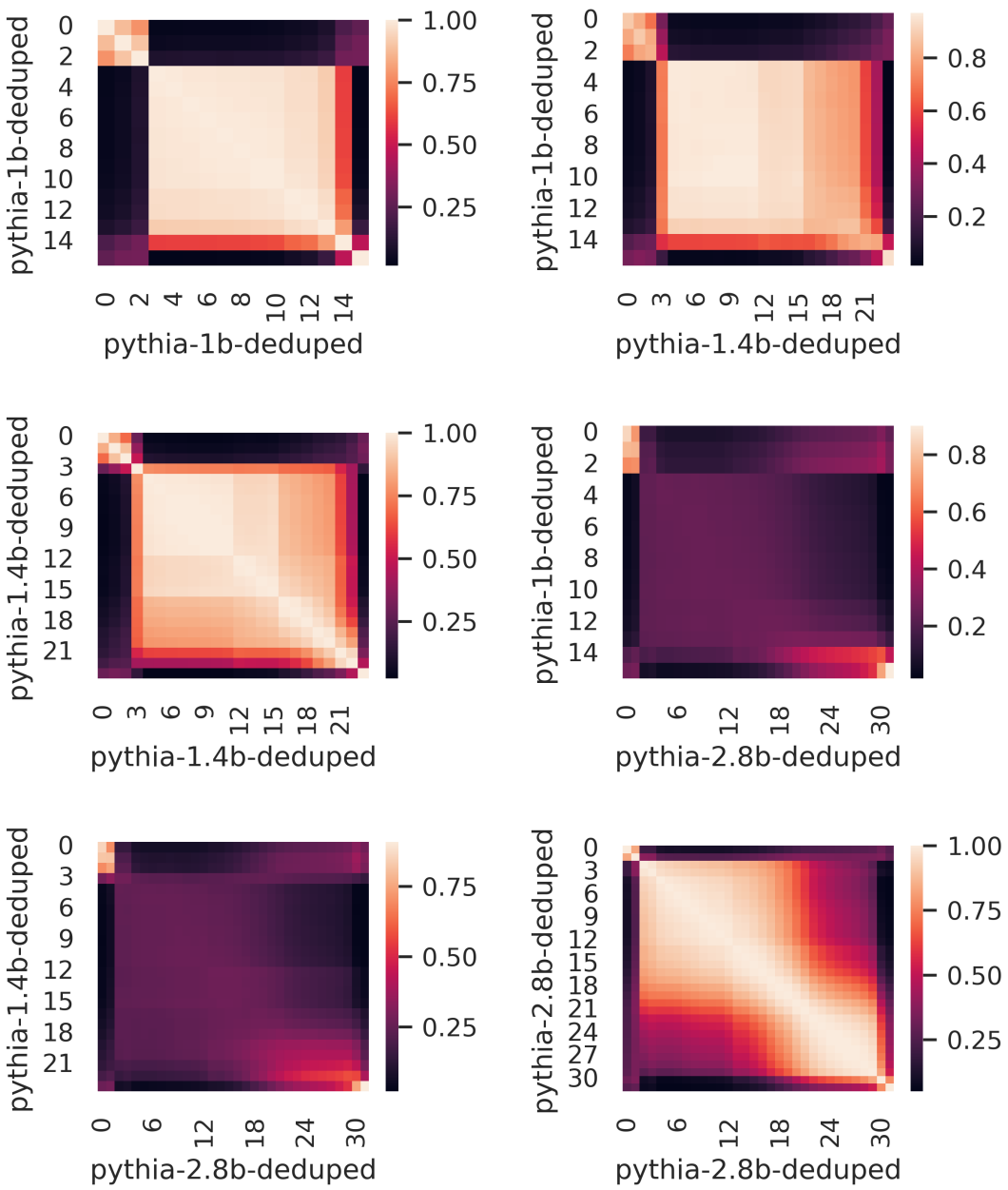

Figure 9: CKA for all pairs of models in the set `pythia-{1b,1.4b,2.8b}`, evaluated on the Pile dataset (Gao et al., 2020) (higher means more similar).

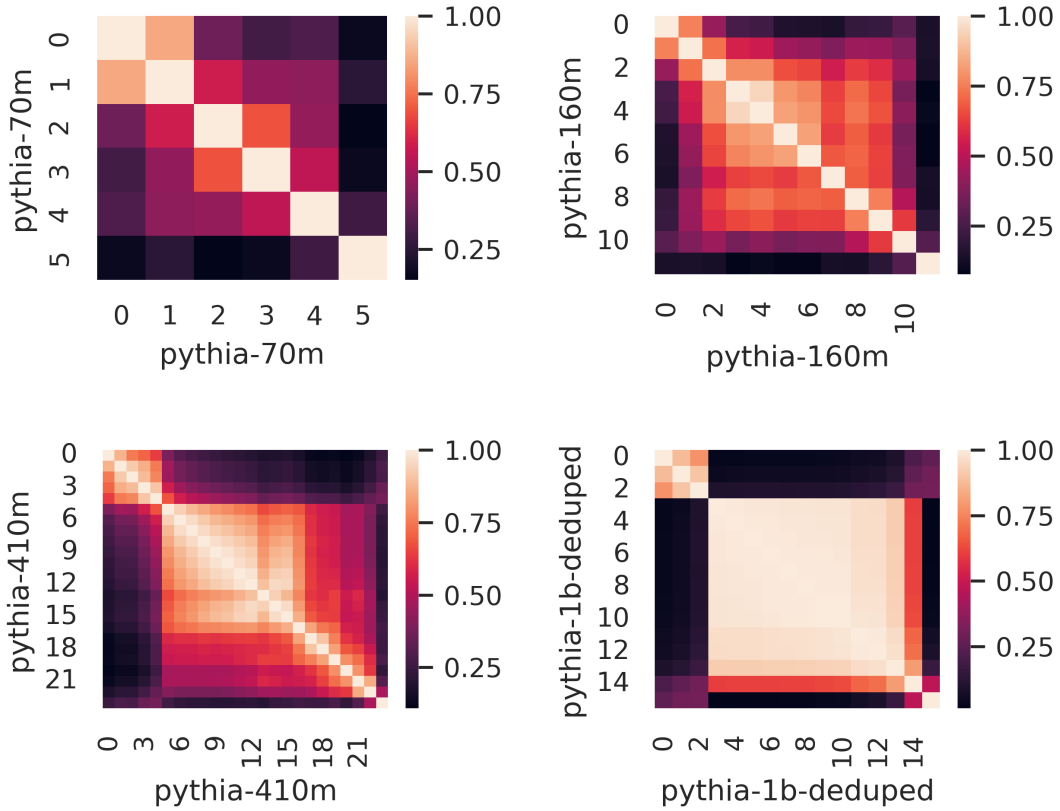

Figure 10: Intra-model CKA for models in the set `pythia-{70m,140m,410m,1b}`, evaluated on the Pile dataset (Gao et al., 2020) (higher means more similar).

C Experimental details

C.1 SoLU/GeLU and Pythia models

For the SoLU-GeLU and Pythia experiments we use the TransformerLens library (Nanda, 2022), whose developers pre-trained the SoLU/GeLU models on the Pile (Gao et al., 2020) (architecture details can be found here and some training details are available here). We use a heavily adapted version of the tuned-lens library to performing stitching (Belrose et al., 2023). In particular, for stitching we learn an affine transformation (along with an additional LayerNorm) φ with the PILE validation set. Figure 6 plots learning curves for GeLU-SoLU stitching layer optimization. We borrow many of the Tuned Lens hyperparameters, and train for 2000 steps (200 warm-up steps) with SGD with Nesterov momentum with a learning rate of 1.0 and cross-entropy loss.

When computing CKA, we extract features after each transformer block (*after* addition with the residual, technically the `hook_resid_post` hook point of TransformerLens). Letting f and g be the two models under consideration and letting X be an input batch of token sequences, this yields feature tensors say

$$H_i = f_{\leq i}(X) \text{ and } H'_j = g_{\leq j}(X)$$

for $i = 1, \ldots, L$ and $j = 1, \ldots, L'$ where L, L' are the respective depths of f and g. In our experiments we use batches consisting of 1 (!) sequence of 1024 tokens (the default context length for the Pythia models); these sequences are constructed by "packing" datapoints from the Pile.[5]. When one or more of f, g is large (e.g. has more than 1 billion parameters), we run the necessary forward passes of f and g on separate GPUs — we used a mix of NVIDIA A100 GPUs with either 40 or 80 GB of RAM.

We then flatten the tensors of shape (batch, sequence, feature) (in our experiments (1, 1024, `hidden_dim`)) to a matrix of features with shape (batch $*$ sequence, feature) (so in our experiments (1024, `hidden_dim`)),[6] and then compute CKA values in batches using an implementation of the unbiased estimator described in (Nguyen et al., 2021). Explicitly, if H_i and H'_j are (already flattened) hidden feature matrices of shape (batch $*$ sequence, feature) (with possibly distinct feature dimensions), we first mean center both matrices ($H_i \leftarrow H_i - \mathbf{1}\mathbf{1}^T H_i$, similarly for H'_j) and compute the kernels

$$K_i = H_i H_i^T \text{ and } L_j = H'_j {H'}_j^T$$

Next, we make use of the unbiased Hilbert-Schmidt independence criterion $\mathrm{HSIC}_1(A, B)$, defined for symmetric matrices A and B of equal shape (n, n) as follows: first, let $\tilde{A}$ and $\tilde{B}$ be obtained by replacing the diagonals of A and B respectively with 0s.[7] Then,

$$\mathrm{HSIC}_1(A, B) := \frac{1}{n(n-3)}\Big(\mathrm{tr}(\tilde{A}\tilde{B}^T) + \frac{1}{(n-1)(n-2)}\mathrm{tr}(\tilde{A}\mathbf{1}\mathbf{1}^T)\mathrm{tr}(\tilde{B}\mathbf{1}\mathbf{1}^T) - \frac{2}{n-2}\tilde{A}\mathbf{1}\mathbf{1}^T\tilde{B}\Big).$$

This unweildy-looking expression can be computed efficiently by noting for example that $\mathrm{tr}(\tilde{A}\tilde{B}^T)$ is simply the dot-product of the vectorizations of $\tilde{A}$ and $\tilde{B}$, $\mathrm{tr}(\tilde{A}\mathbf{1}\mathbf{1}^T)$ is just the sum of all entries of $\tilde{A}$, and $\tilde{A}\mathbf{1}\mathbf{1}^T\tilde{B}$ is just the dot product of the vectors obtained by summing columns of $\tilde{A}$ and $\tilde{B}$.

Finally, we compute a matrix S of shape (L, L') with entries

$$S_{ij} = \frac{\mathrm{HSIC}_1(K_i, L_j)}{\mathrm{HSIC}_1(K_i, K_i) \cdot \mathrm{HSIC}_1(L_j, L_j)},$$

and average these batch-estimated matrices S over many randomly sampled batches, in our experiments 1000.

[5]Explicitly, we use the `chunk_and_tokenize` function from TransformerLens

[6]In other words, we are comparing activations at the token level. An alternative which we have not yet evaluated would be flattening the sequence and feature indices to a single dimension and comparing activations at the sample level.

[7]In PyTorch or NumPy, accomplished with $\tilde{A} = A - \mathrm{diag}(\mathrm{diag}(A))$.